/crew:autopilot

This guide is task-first. Each section answers one question you will have while autopilot drives a ticket: how to switch it on, how to hand it work, what its output means, and why it stopped. Every sample of script output below was printed by the script itself in a throwaway repository; the paths in them are shortened to `/path/to/repo`. Features that have not landed yet are listed once, under "What is coming", with their ticket ids and no invented output.

What Autopilot Is

- It drives **one ticket** through spec, plan, approval, implement, refresh, review, done and ship.
- It follows each phase's own command (`/crew:spec`, `/crew:plan`, `/crew:implement`, ...) in the same session, exactly as if you had typed it.
- It does not choose the order. Every turn it asks `crew_autopilot.py next`, which reads the ticket's files on disk and names the next phase, so a skipped phase is visible.
- It stops wherever a person is needed: a direction to approve, review findings with a BLOCK, an action on the never-without-a-yes list, or anything it cannot tell.
- It is **off by default**. Nothing happens until you arm it.

Arm It

Autopilot reads `.crew/config.json` in the repository. Only the exact string `plan` arms it:

```
{
  "autopilot": { "mode": "plan", "maxPhases": 12 }
}
```

`maxPhases` is how many phases one invocation may run before it stops and reports (default 12). Check what is in force:

```
python3 -B plugin/crew/hooks/scripts/crew_autopilot.py settings --root .
```

In a repository where crew is installed as a plugin, the script is under the plugin's `hooks/scripts/`

directory; `/crew:autopilot` runs it for you. A fresh repository with only `"mode": "plan"` set prints:

```
mode=plan maxPhases=12 deploy=none maxAutoReplans=0

approval=risk questions=risk

sleep=off schedule=none approval=- questions=- source=schedule
```

- **Line 1** is what autopilot may do: armed (`mode=plan`), at most 12 phases per run, no unattended deploy (`deploy=none`), no automatic replanning after a review runs out of rounds (`maxAutoReplans=0`).
- **Line 2** is who decides an approval and an open question: `risk` for both (see "Approvals").
- **Line 3** is the sleep schedule: off, so the day values on line 2 apply all the time.
- Any `warning:` line below them names a setting crew read as something else, and why.

Hand It Work

You type	What happens
`/crew:autopilot`	picks the ticket (below) and drives it
`/crew:autopilot <id>` or `/crew:autopilot run <id>`	drives that ticket
`/crew:autopilot status`	reports where the ticket stands; runs nothing

Which ticket, when you name none. In this order: the handoff's `resume:` line (only when its `branch:` and `head:` match this checkout), then this worktree's active ticket, then `.work/INDEX.md` only when exactly one ticket is open. Autopilot prints where the ticket came from and every source it fell through. It never picks from a "next action" list.

Work that has no ticket yet. Write the request as a direction file under `.work/autopilot/` and mint a ticket from it:

```
python3 plugin/crew/hooks/scripts/crew_ticket.py assign --root . --direction-file
.work/autopilot/csv-export.md
```

`crew_ticket.py assign` refuses, minting nothing, when the file is not under `.work/autopilot/`, when autopilot is not armed, or when the direction is missing a section. It writes the ticket as `ready` with the direction you gave; approval still goes through the policy below. The `/crew:autopilot assign`

command that does this from plain text has not landed (see "What is coming").

Read Its Output

`/crew:autopilot Status`

`status` is read-only, armed or not. For a ticket whose direction is approved and which has no spec yet:

```
mode: plan, maxPhases 12

ticket: ABC-510 (from .work/INDEX.md)

phase: spec - next: /crew:spec ABC-510

waiting on: autopilot - run /crew:autopilot to continue

review: EMPTY, 2 of 2 rounds left

resume: no .work/HANDOFF.md

fell through: no .work/HANDOFF.md

fell through: no active-ticket entry for this worktree
```

Line	Meaning
`mode:`	armed or not, and the phase budget
`ticket:`	which ticket, and where it was found
`phase:`	the next phase and its command, or `stopped` with the reason
`waiting on:`	who has to act: `autopilot` (run it again), or `owner` (you) and what to type
`review:`	the review ledger: no round yet, and how many rounds are left
`resume:`	what the handoff file says, if there is one
`fell through:`	each ticket source that did not answer, in order

`unknown` anywhere means crew could not tell, never "nothing to do". With the plan written but not approved, the same ticket reads:

```
phase: approve, stopped - ABC-510 has no approved plan. The human types /crew:approve
ABC-510. status reads no approval policy: crew_autopilot.py settings shows whether
autopilot.approval lets /crew:autopilot approve it instead

waiting on: owner - types /crew:approve ABC-510
```

The `Next` Line

Every turn autopilot runs `crew_autopilot.py next --root . --ticket <id>` and gets one line:

```
phase=spec stop=0 command=/crew:spec ABC-510 reason=no spec.md
```

- `phase` - the phase it names from disk.
- `stop` - `0`: run `command` now. `1`: stop and tell you `reason` and the command you type.
- `command` - the phase command, or empty when only you can act.
- `reason` - why, in words.

No output, a traceback or a non-zero exit is a stop, never a guess.

"Autopilot Is Armed ... Its First Phase Is Spec"

When you start autopilot on a ticket whose direction you approved, the session reports something like this (it is the session's report, not a line a script prints):

> Autopilot is armed (mode=plan, maxPhases 12) and has activated ABC-510. Its first phase is spec: /crew:spec ABC-510

That is **not a stop**. With an approved direction and no `spec.md`, `next` answers `phase=spec stop=0`, so autopilot runs `/crew:spec ABC-510` straight away. The two phases where it stops before any work are `brainstorm` (no direction yet) and `direction-approval` (a direction that waits for your yes). For the same ticket while its INDEX row still says `direction`:

```
phase=direction-approval stop=1 command= reason=INDEX.md status is `direction`: direction.md
waits for the owner's yes in /crew:brainstorm
```

Approvals

One ticket. Type `/crew:approve ABC-510`. The approval records a hash of `spec.md` and `plan.md`; any later edit to either, other than the header's status value, makes it **stale**, and autopilot stops at `approve` again until you re-approve.

Several tickets at once. `/crew:approve T-4 T-5`, or a range `/crew:approve T-0010..T-0012`. crew shows a PENDING list with each ticket's plan and spec hash and records nothing until you type `/crew:approve --confirm` as your next prompt.

Letting autopilot approve. `autopilot.approval` and `autopilot.questions` each take one of three policies:

Policy	Approval of a plan	An open question
`human`	always stops for you	always stops for you
`self`	autopilot approves	autopilot takes the recommended option
`risk` (default)	autopilot approves only a spec whose header says `risk: low`	the same rule, by the ticket's risk

A missing or unreadable risk counts as `high`. Every `autopilot.approval` setting also needs `autopilot.mode: "plan"` (or `"backlog"`) and `scope.allowCliApproval: true` in `.crew/config.json` (the latter defaults to `false` and is a hand edit by the owner), so out of the box nothing self-approves (`autopilot.questions` does not need either). A refusal lists every unmet one at once, not only the first, each with its value as written (a typo such as `"plna"` is shown, with a note that it is not a valid value) and capped at 40 characters; a config file that cannot be read is reported as "could not be told", never as a value, and names which file (repo or machine-global, with its path) is unreadable. Under `risk`, a spec that is not `risk: low` (or has no risk header) is listed too, with what the policy needs. When the machine-global file also sets `autopilot.mode` or `autopilot.approval`, the line says which file holds which value and which to edit. It names the file to hand-edit, which in a lane worktree is the main checkout's. Refused:

```
refused: scope.allowCliApproval is not exactly true in .crew/config.json, so no approval but the human's counts; the human types /crew:approve ABC-510

For autopilot to approve plans itself ALL of these must hold; unmet now:
```

```
  - scope.allowCliApproval is false, needs true (a hand edit of
/path/to/repo/.crew/config.json, the file this checkout reads, by the owner: "scope":
{"allowCliApproval": true}; crew_config.py --set refuses it)
```

Allowed (`"approval": "self"` and `"scope": {"allowCliApproval": true}`):

```
self-approved ABC-510 under approval=self, risk=low

phase=implement stop=0 command=/crew:implement ABC-510 reason=approved, no review round
under this plan
```

Autopilot reports every `self-approved` line by name in its final report. A ticket minted by `crew_ticket.py assign` follows the same policy as any other.

Overnight. `autopilot.sleep.schedule` (`"HH:MM-HH:MM"`, local time, default off) names one nightly window in which `autopilot.sleep.approval` and `autopilot.sleep.questions` replace the two day values, so an overnight run can approve where the day setting would stop. `/crew:autopilot sleep` starts sleep mode now and `/crew:autopilot wake` ends it; both need `scope.allowCliApproval: true`, and until L-1504 a manual sleep only ever tightens a policy. A plan approved asleep stops standing when the window ends if the day policy would not have approved it. Without the setting:

```
refused: scope.allowCliApproval is not exactly true in .crew/config.json, so only the
schedule puts autopilot to sleep
```

What It Stops For

Always a person. `brainstorm` (no approved direction) and `review-acceptance`: review findings with any BLOCK, or a round the ledger's guarded accept refuses. A BLOCK is never accepted by autopilot, at any setting. The one way past a BLOCK without you is a new plan, and it is off by default: see "Review" below.

Enforced by `next` from disk, every turn. `needs-replan`, `needs-replan-or-revert`, `unknown-ledger`, `failed-validate`, `direction-unknown`, `index-disagreement`, `unsettled-artifact`, `ticket-mismatch`, `max-phases`, `no-progress`, `auto-replan-cap` and `drift`, plus, when it runs with `--runner`, `in-flight` (another runner holds the ticket here, or its marker is stale or unreadable) and `handover-elsewhere` (a live holder drives it from another worktree). `crew_autopilot.py stops --root .` lists them from the code.

Never without an explicit yes. Offboarding a role (`offboard-role`), deleting a code map or diagram (`delete-map`), rewriting the repository's crew metrics file (`rewrite-metrics`), and any force-push, branch

delete, history rewrite or removal of a tracked file (`git-destruction`), and clearing another runner's in-flight marker (`clear-inflight`).

Deploys. `autopilot.deploy` is `none` (default), `nonprod` or `all`. Production needs `all` **and** `environments.prodUnattended: true` in both config layers, with the cloud guard in `block`. `crew_autopilot.py deploy-allowed` answers for one environment:

```
verdict=allow env=staging class=nonProd reason=autopilot.deploy=nonprod allows nonProd

verdict=ask env=prod class=prod reason=autopilot.deploy is nonprod; production needs all
```

Nothing in this crew version dispatches a deploy itself; `settings` warns you of that when the key is not `none`.

Being told it stopped. With crew's notifications configured (Telegram or Teams, in the machine-global config), a ping is sent when Claude Code stops and waits on you, led by a subject such as `Question` and naming the ticket and what it waits on; there is no ping per phase.

Review. A review phase ends at its verdict: autopilot reports the BLOCK and FIX lines verbatim and goes back to `next`. Fixing, accepting and replying on the PR are yours, with two exceptions. After the ledger says `review: auto-accept: eligible` on a final round with no BLOCK, autopilot runs `review_ledger.py --auto-accept`, the guarded accept that refuses anything it cannot read cleanly. And with `autopilot.maxAutoReplans` at 1 or more (default 0, off), a final round with a BLOCK that has no review round left can be rejected by autopilot itself (`crew_autopilot.py auto-reject`, which prints an `auto-rejected` line and every BLOCK and FIX line): `/crew:plan` then writes a successor plan that quotes each of them, and the approve phase decides it under `autopilot.approval`. The BLOCK is never accepted; once `maxAutoReplans` successor plans are on the ledger, autopilot stops at `auto-replan-cap`.

Ship. After `/crew:done`, `crew_autopilot.py ship` pushes the branch (never force), opens the PR, and under `autopilot.ship: merge` (the default; `pr` stops at the open PR) merges it with a merge commit only when every required check passes and the review receipt still stands. A `high`-risk ticket never merges on same-family reviews. It prints `action=... stop=... pr=... reason=....`

Resume After /clear

When context runs low, autopilot writes a handoff with a resume line:

```
resume: /crew:autopilot ABC-510
```

After `/clear` (or in a new session), type `/crew:autopilot` with no id: the handoff's `resume:` line names the ticket, but only while its `branch:` and `head:` match this checkout. A handoff from another branch or an older commit falls through to the active ticket, and `status` says so on a `fell through:` line. With auto-resume configured, crew can also type that resume command into its own session for you after the context cycle.

Plain text. With `route.enabled: true` in `.crew/config.json` (off by default), a whole prompt of `continue`, `keep going` or `carry on` routes to whatever `next` names for the ticket, and `autopilot status` or `what is autopilot doing` routes to `/crew:autopilot status`, and `heading to bed` or `I'm back` to `/crew:autopilot sleep` or `wake`. A `continue` whose next step is an approval asks instead: no phrase ever approves.

Command Table

Command	Status	What it does
`/crew:autopilot`, `/crew:autopilot <id>`, `/crew:autopilot run <id>`	landed	drive one ticket
`/crew:autopilot status`	landed	report, read-only
`/crew:autopilot focus <id>`, `focus`, `focus off`	landed	lock autopilot onto one ticket, show it, release it
`/crew:autopilot sleep`, `/crew:autopilot wake`	landed	start or end sleep mode now (needs `scope.allowCliApproval: true`)
`/crew:autopilot assign`	arrives with L-0611 (the router's message still names T-0019)	mint a ticket from plain text
`/crew:autopilot goal`	arrives with T-0012	work towards a goal across tickets
`crew_autopilot.py settings`	landed	the settings in force
`crew_autopilot.py status`	landed	the status lines above
`crew_autopilot.py next`	landed	the one `next` line

Command	Status	What it does
`crew_autopilot.py resume`	landed	which ticket, and from where
`crew_autopilot.py route`	landed	how the command's arguments are read
`crew_autopilot.py approve`	landed	self-approval under the policy, or `refused:`
`crew_autopilot.py questions-check`	landed	checks a researched answer before it is taken
`crew_autopilot.py stops`	landed	every stop, from the code
`crew_autopilot.py deploy-allowed`	landed	allow, ask or refuse for one environment
`crew_autopilot.py ship`	landed	push, PR and guarded merge after `/crew:done`
`crew_autopilot.py auto-reject`	landed	under `maxAutoReplans`, reject an out-of-rounds BLOCK round for a successor plan
`crew_autopilot.py focus`, `sleep`, `wake`	landed	the writers behind the three commands above

What Is Coming

Not landed in this crew version. No output is shown for any of them; each ticket adds its own section here when it lands.

Example	Ticket	Phrase
Approving a goal's split, `goal:<slug>`	T-0012	`goal:<slug>`
`/crew:autopilot assign`, from plain text	L-0611	`assign`
A goal across several tickets	T-0012, L-0541	`goal`
A wave: tickets run in parallel	T-0029	`wave`

Example	Ticket	Phrase
Autopilot's own split into PR slices	T-0052, T-0058, T-0059	`split`
A goal that survives /clear, a branch switch and a crash (`--goal`)	T-0056	`--goal`
The full goal walkthrough	T-0012, L-0541	`walkthrough`

When Something Looks Wrong

- **"It stopped at spec, but I wanted it to keep going."** It did not stop there: `phase=spec stop=0` runs. Read the `next` line; a real stop has `stop=1` and a reason.
- **"It will not approve my plan."** `scope.allowCliApproval` is not `true`, or the policy is `human`, or the ticket's risk is above what `risk` allows. The `refused:` line says which.
- **"It keeps stopping at approve after I edited the spec."** The approval went stale. Re-approve.
- **"It stopped at review-acceptance."** A BLOCK, or a round the guarded accept refused. That is yours to fix or accept.
- **"`unknown` in status."** crew could not read something it needed; the same line says what.
- **Stale installs, hook noise, review loops and turning things off** are in the troubleshooting guide, "crew - Troubleshooting".